CASE STUDY 1

MONEY AND PRICING IN AN E-COMMERCE CATALOG

Q1. Why is double unsuitable for representing money?

The double data type uses binary floating-point representation. Many decimal fractions, such as 0.1, cannot be represented exactly in binary. As a result, calculations involving money may contain small rounding errors.

- Repeated additions and subtractions can accumulate floating-point errors.
- The cart total and invoice total may differ by a small amount.
- A decimal amount can be stored as an integer in the currency's smallest unit (minor unit). For example, USD 25.75 is stored as 2575 cents, while EUR 10.50 is stored as 1050 cents.
- Integer minor units avoid binary floating-point drift. The system must still define a rounding rule, such as HALF_EVEN, for calculations that produce fractions of a minor unit.

Conclusion: Integer minor units provide exact storage for monetary amounts, while a documented rounding rule ensures consistent results.

Q2. Classify Money, Product, Cart, Percentage and Quantity.

An entity has a distinct identity that remains the same even when its properties change. A value object is identified by its values; two objects with the same relevant values are equal.

Concept	Classification	Reason
Money	Value Object	Two amounts with the same value and currency are interchangeable.
Product	Entity	A product is identified by its SKU, even if its name or price changes.
Cart	Entity	A cart has a unique Cart ID and its contents can change over time.
Percentage	Value Object	A percentage is identified by its numerical value.
Quantity	Value Object	A quantity is identified by its positive whole-number value.

Therefore, Product and Cart are entities, while Money, Percentage and Quantity are value objects.

Q3. Design a Percentage Value Object.

A Percentage value object should accept only values from 0% to 100%, provide an applyTo(Money) operation, and compare objects by value. The following Java example uses basis points, where 10,000 basis points represent 100%.

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

enum Currency {
    USD(2), EUR(2), JPY(0), KWD(3);

    final int minorUnits;

    Currency(int minorUnits) {
        this.minorUnits = minorUnits;
    }
}

final class Money {
    private final long minor;
    private final Currency currency;

    private Money(long minor, Currency currency) {
        if (minor < 0) {
            throw new IllegalArgumentException("Negative amount");
        }
        this.minor = minor;
        this.currency = Objects.requireNonNull(currency);
    }

    public static Money of(String amount, Currency currency) {
        BigDecimal value = new BigDecimal(amount);
        if (value.signum() < 0) {
            throw new IllegalArgumentException("Negative amount");
        }

        long minor = value
            .setScale(currency.minorUnits, RoundingMode.UNNECESSARY)
            .movePointRight(currency.minorUnits)
            .longValueExact();

        return new Money(minor, currency);
    }

    public long getMinor() {
        return minor;
    }

    public Currency getCurrency() {
        return currency;
    }

    @Override
    public String toString() {
        return BigDecimal.valueOf(minor, currency.minorUnits)
            + " " + currency;
    }
```

```
}

final class Percentage {
    private final int basisPoints;

    // Private constructor: objects are created through validated factory methods.
    private Percentage(int basisPoints) {
        if (basisPoints < 0 || basisPoints > 10000) {
            throw new IllegalArgumentException(
                "Percentage must be between 0 and 100"
            );
        }
        this.basisPoints = basisPoints;
    }

    public static Percentage of(int wholePercent) {
        if (wholePercent < 0 || wholePercent > 100) {
            throw new IllegalArgumentException("Invalid percentage");
        }
        return new Percentage(wholePercent * 100);
    }

    public static Percentage ofBasisPoints(int basisPoints) {
        return new Percentage(basisPoints);
    }

    // Returns the monetary amount represented by this percentage.
    public Money applyTo(Money money) {
        BigDecimal result = BigDecimal.valueOf(money.getMinor())
            .multiply(BigDecimal.valueOf(basisPoints))
            .divide(BigDecimal.valueOf(10000), 0, RoundingMode.HALF_EVEN);

        BigDecimal decimalAmount = result.movePointLeft(
            money.getCurrency().minorUnits
        );

        return Money.of(decimalAmount.toPlainString(), money.getCurrency());
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Percentage)) return false;
        Percentage other = (Percentage) obj;
        return basisPoints == other.basisPoints;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(basisPoints);
    }

    @Override
    public String toString() {
        return (basisPoints / 100.0) + "%";
    }
}

public class MoneyPricingDemo {
```

```
    public static void main(String[] args) {
        Money price = Money.of("100.00", Currency.USD);
        Percentage discount = Percentage.of(10);

        Money discountAmount = discount.applyTo(price);

        System.out.println("Original Price: " + price);
        System.out.println("Discount: " + discount);
        System.out.println("Discount Amount: " + discountAmount);
    }
}
```

Expected output:

```
Original Price: 100.00 USD
Discount: 10.0%
Discount Amount: 10.00 USD
```

Explanation: The private constructor prevents direct creation of an invalid Percentage. The factory methods validate values before creating an object. applyTo() calculates the percentage amount using decimal arithmetic and HALF_EVEN rounding. equals() compares the basis-point value, and hashCode() uses the same value so the equality/hash-code contract is maintained.

Q4. Should Money.plus() silently convert currencies?

No. Money.plus() should not silently convert one currency into another. It should add amounts only when both Money objects use the same currency.

- Purity: Addition should depend only on its input values. A conversion based on today's exchange rate depends on changing external data, so the operation is no longer predictable.
- Testability: Tests become difficult to reproduce if exchange rates change. A separate conversion service can receive a fixed rate and date, making tests deterministic.
- Invariants: The model states that money in different currencies cannot be added or compared directly. Silent conversion hides a currency mismatch and weakens this guarantee.
- Transparency: Currency conversion should be an explicit business operation so developers can see which rate and date were used.

Recommended design: Keep Money.plus() restricted to matching currencies. Use a separate currency-conversion service that accepts the source amount, target currency, exchange rate and conversion date, and returns a new Money value.

Conclusion: Separating addition from currency conversion preserves clear invariants and improves purity and testability.

Q5. How should the model support refunds without allowing negative product prices?

The model should distinguish a product's selling price from a general financial amount. ProductPrice should allow only non-negative values, while SignedMoney should allow positive, zero or negative values for refunds and adjustments.

1. ProductPrice

- Represents the selling price of a product.
- Can be created only from a non-negative Money value.
- Prevents a product from being assigned a negative price.

```
final class ProductPrice {
    private final Money amount;

    private ProductPrice(Money amount) {
        this.amount = Objects.requireNonNull(amount);
    }

    public static ProductPrice of(String value, Currency currency) {
        return new ProductPrice(Money.of(value, currency));
    }

    public Money getAmount() {
        return amount;
    }
}
```

2. SignedMoney

- Represents a monetary amount that may be positive, zero or negative.
- Can be used for refunds, credits and financial adjustments.
- Must retain its currency and reject arithmetic between different currencies.

```
final class SignedMoney {
    private final long minor;
    private final Currency currency;

    public SignedMoney(long minor, Currency currency) {
        this.minor = minor;
        this.currency = Objects.requireNonNull(currency);
    }

    public SignedMoney plus(SignedMoney other) {
        if (currency != other.currency) {
            throw new IllegalArgumentException("Currency mismatch");
        }
        return new SignedMoney(
            Math.addExact(minor, other.minor), currency
        );
    }

    @Override
    public String toString() {
        return minor + " minor units " + currency;
    }
}
```

For example, a product may have a ProductPrice of USD 500.00, while a refund may be represented as SignedMoney with a value of -10,000 cents (USD -100.00). The product-price type remains non-negative, while the signed type supports refunds.

Conclusion: Using separate types preserves the product-price invariant while allowing the business to represent refunds and other negative financial adjustments.

Conclusion

This case study demonstrates how object-oriented modelling can make an e-commerce pricing system safer and more reliable. Integer minor units avoid floating-point drift; entities and value objects represent identity and equality correctly; validated immutable objects prevent invalid discounts and quantities; and explicit currency rules prevent incorrect additions. Separating ProductPrice from SignedMoney also supports refunds without allowing negative product prices.